

Низкоуровневое проектирование (LLD): компоненты и взаимодействия

AI Архитектор



Проверить, идет ли запись

Меня хорошо видно & слышно?



Тема вебинара

ML для финансового анализа

Course project



Игорь Стурейко

Руководитель курсов: Reinforcement Learning, ML Professional, ML Basic, MLOps, FinML

Teamlead, главный инженер проекта,
Физический факультет МГУ, PhD теоретическая физика

Опыт:
Более 20 лет занимался прикладной математикой и мат моделированием
(Data Scientist) (Python, C++)

@stureiko (TG)

LinkedIn: [igor-stureiko](#)

@rl_fintech (Мой канал посвященный финансовым моделям)



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Карта курса



СТАРТ

Стратегический фундамент
и планирование проекта

Проектирование и
документирование архитектуры

Продвинутые
архитектурные
паттерны

Инфраструктура

Качество, интеграции
и безопасность

Стратегия, лидерство и
экономика

Проектная работа

ФИНИШ



Маршрут вебинара

Знакомство

Какие бывают архитекторы?

C4 модель представлений (Level 3&4)

C4 на примере (продолжение)

API & Contracts

Анти-паттерны C4

Цели вебинара

К концу занятия вы сможете

детализировать высокоуровневые
компоненты до уровня конкретных
классов и модулей

проектировать API и протоколы
взаимодействия между сервисами

Смысл

Для чего вам это уметь

для того чтобы детализировать
постановку для вашей команды



**Занимался ли кто-нибудь
из вас низкоуровневым
проектированием?**



**Какие бывают
архитекторы?**

Сравнение архитекторов

	 Software architect	 Solution architect	 Enterprise architect
Среда обитания	Среди разработчиков	Команда ИТ-проекта	Башня из слоновой кости :)
Инструмент	Интегрированная среда разработки	Whiteboard, Enterprise wiki	EA Tools, PowerPoint
Нотация моделирования	ERD, UML class, component diagram	Boxes and arrows	Archimate
С кем взаимодействуют	Разработчики, тестировщики	Разработчики, администраторы, заказчики	Топ-менеджеры, руководители
Что обсуждают, о чем говорят	Шаблоны проектирования	Варианты реализации	Реализация стратегии

Зоны ответственности ролей

Артефакт/ процесс	C-level / бизнес	Аналитик	Enterprise	Solution	Software	DevOps / MLOps	Dev
Бизнес-требования	A	R	I	C	I	I	I
Системный ландшафт, стратегия	A	I	R	C	I	C	I
Функциональный требования	A	R	I	C	C	I	I
Нефункциональные требования	I	S	C	A	R	C	S
HLD (C1 и C2)	I	I	C	A / R	C	C	I
LLD (C3 и C4)	I	I	I	I	A / R	C	S
Deployment	I	I	I	I	A / R	C / S	I

R (Responsible): Исполнитель (делает работу).

A (Accountable): Ответственный (принимает результат, «головой отвечает»).

S (Support): Помощник (предоставляет ресурсы/код).

C (Consulted): Эксперт (с кем обязаны посоветоваться).

I (Informed): Наблюдатель (кого уведомляют).



C4 модель представлений (Level 3&4)

Компонент – это

группа функциональности, которые объединены и спрятаны (инкапсулированы) за строго определенный интерфейс.

Например:

- В C++, Java компонентом будет некоторое множество классов и интерфейсов
- В JS компонентом может быть модуль
- В Haskell компонентом тоже может быть группа типов и функций, объединенных в модуль и т.д.



Диаграмма компонент

- Диаграмма показывает содержимое одного контейнера
- Диаграмма отвечает на следующие вопросы:
 - Из чего состоит приложение?
 - Как контейнеры взаимодействуют друг с другом?
- Основные элементы - компоненты
- Дополнительные элементы - контейнеры, люди и внешние системы непосредственно взаимодействующие с компонентами
- Целевая аудитория - архитекторы, разработчики

<https://c4model.com/diagrams/component>

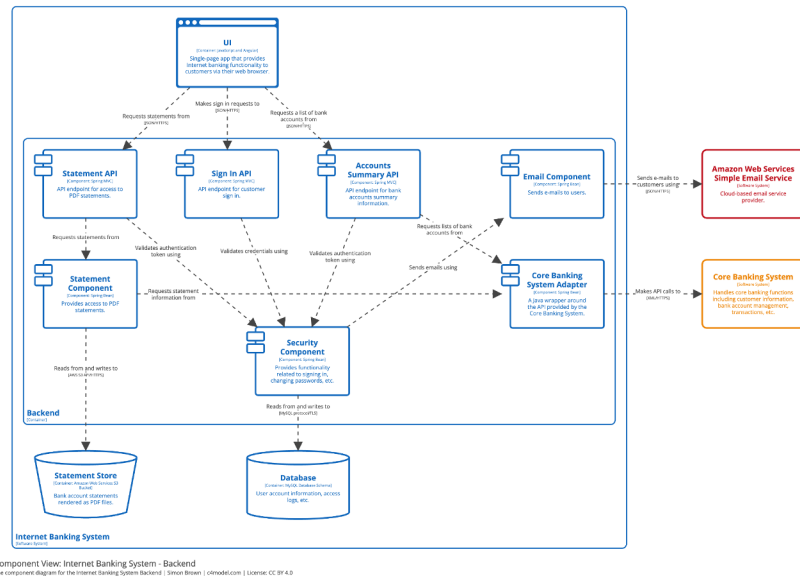
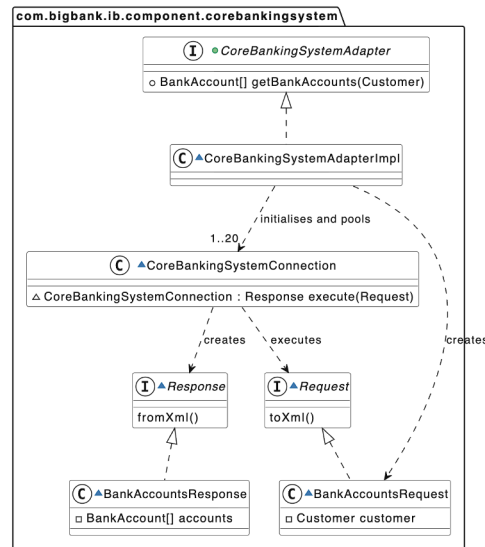


Диаграмма кода

- Диаграмма показывает детали компонента
- Диаграмма отвечает на следующие вопросы:
 - Как элементы кода взаимодействуют друг с другом?
- Основные элементы - элементы кода (классы, интерфейсы)
- Целевая аудитория - архитекторы, разработчики



Code View: Internet Banking System - Backend - Core Banking System Adapter

A summary of the implementation details for the Core Banking System Adapter component | Simon Brown | c4model.com | License: CC BY 4.0

Узел развертывания – это

элемент, на котором запущен инстанс системы/контейнера

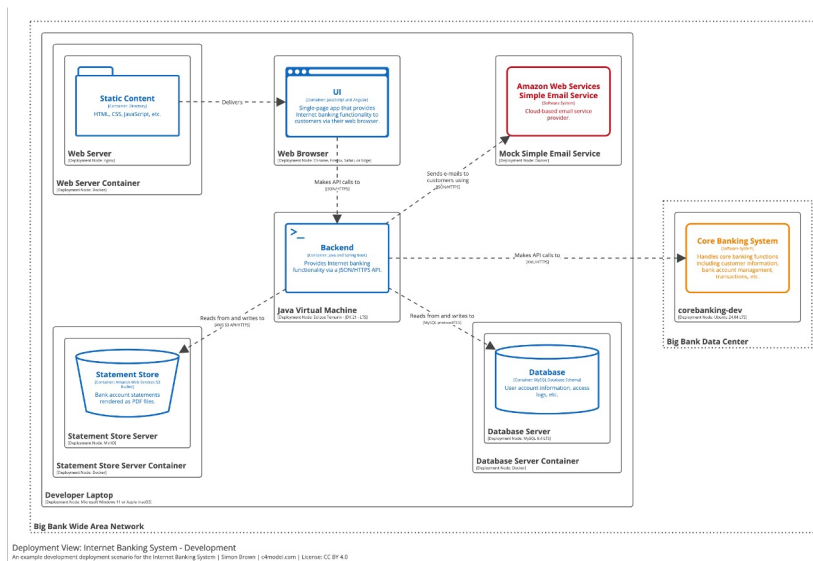
Например:

- физическая инфраструктура (например, сервер)
- виртуальная инфраструктура(например, IaaS, PaaS, виртуальная машина)
- контейнеризированная инфраструктура (например, Docker контейнер)
- окружение для запуска (например, сервер БД, Java EE web/application сервер, Microsoft IIS)



Диаграмма развертывания

- Одна или несколько систем, развернутые в одном окружении
- Диаграмма отвечает на следующие вопросы:
 - Как составные части системы развернуты на узлах развертывания и инфраструктуры?
- Основные элементы - узлы развертывания, инстансы систем и контейнеров
- Дополнительные элементы - узлы инфраструктуры



<https://c4model.com/diagrams/container>

C4 на примере

Внутренний Чат-бот. Бизнес-требования

- Продукт: Внутренний Бот для операторов колл-центра. Поиск по базе знаний.
- Текущая проблема: Операторы тратят 5+ минут на поиск ответа. Клиенты висят на линии.
- Бизнес-цель: Сократить время поиска до 30 секунд.
- Жесткие ограничения (Constraints):
 - стек команды: Python (Middle), опыт в ML — начальный.

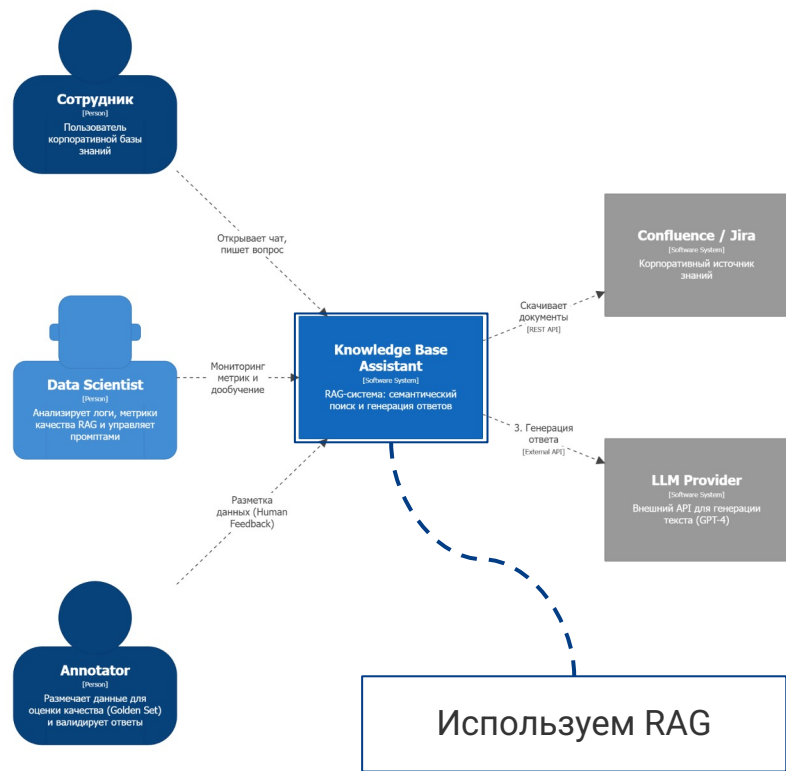


Внутренний Чат-бот. Первые архитектурные решения

Тип	Требование	Архитектурное решение
Business	Бот должен отвечать только по внутренней базе знаний (без галлюцинаций).	Использование паттерна RAG (поиск + генерация) вместо дообучения.
NFR	Время ответа пользователю должно быть < 3 секунд.	Асинхронная загрузка документов, чтобы не блокировать поиск.
Security	Персональные данные клиентов не должны уходить во внешнего провайдера	Фильтрация перс.данных перед отправкой во внешних провайдеров



Чат-бот. C4 – Level 1

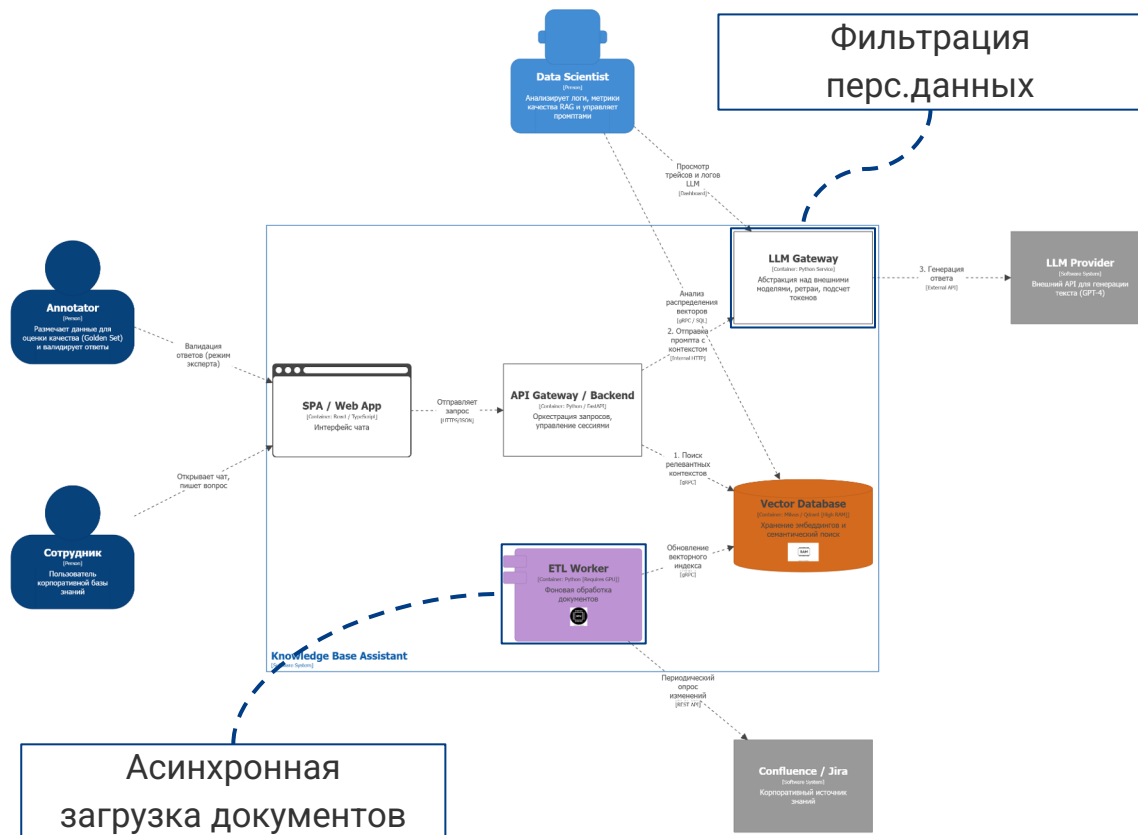


C4 Level 1: System Context
Общая схема взаимодействия пользователя и систем.
четверг, 8 января 2026 г. в 12:51 Москва, стандартное время

- В системе у нас есть и классические пользователи, и новые AI специфичные роли (Data Scientist, Annotator), которые взаимодействуют с продакшном так же часто, как и конечные клиенты. Если их не нарисовать на L1, мы забудем про их доступы при проектировании безопасности (IAM).
- Мы явно визуализируем процесс контроля качества через Аннотатора. Это архитектурное подтверждение того, что система не полностью автономна и требует валидации.
- Добавление технических ролей сразу ставит вопрос: "А как Data Scientist попадает в контур?" (VPN, Bastion host?). Это переводит разговор от абстракции к реальной сетевой безопасности.



Чат-бот. C4 — Level 2



- Контейнеры в AI неравнозначны. Пометки [GPU] и [High RAM] — это сигнал для девопсов и бюджета.
- Мы изолируем ETL Worker, чтобы тяжелая математика не "положила" API для пользователей.
- LLM Gateway — это не просто прокси. Это архитектурное место для PII-фильтрации (анонимизации данных) перед отправкой во внешний мир.

Внутренний Чат-бот. Принимаем решения дальше

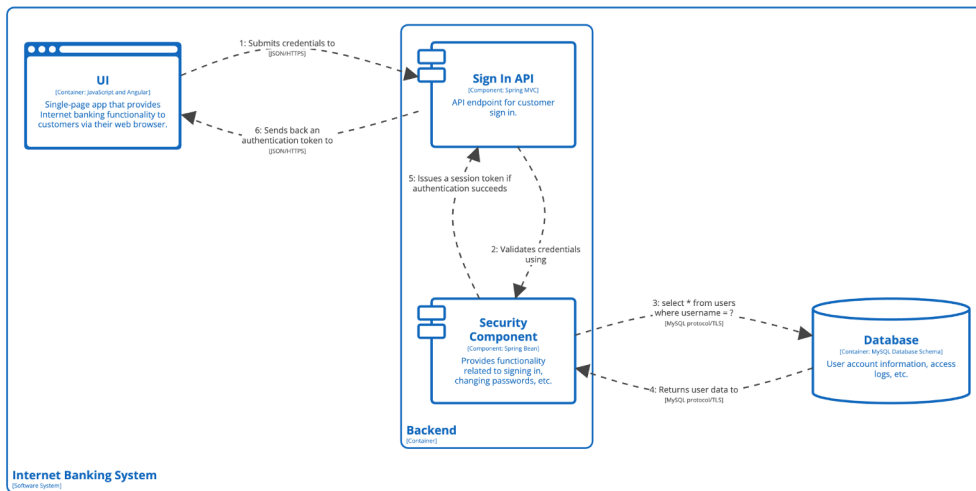
Тип	Требование / Ограничение	Проблема	Архитектурное решение
Maintainability	Поддержка разных форматов (PDF, Docx, Wiki)	Нужна поддержка разных форматов документов. Логика парсинга часто меняется.	Нужен фасад для библиотек парсинга
NFR	Ограничение контекста LLM (например, 8k токенов)	Мы не можем засунуть книгу целиком в вектор.	Нужно нарезать данные библиотеки на чанки
Business	Семантический поиск (искать по смыслу, а не словам)	База данных не понимает текст, ей нужны векторы.	Нужно использовать векторное сравнение данных
NFR	Время ответа пользователю должно быть < 3 секунд.	Разные типы нагрузок нужно изолировать	Используем разные неймспейсы



Динамическая диаграмма (UML collaboration)

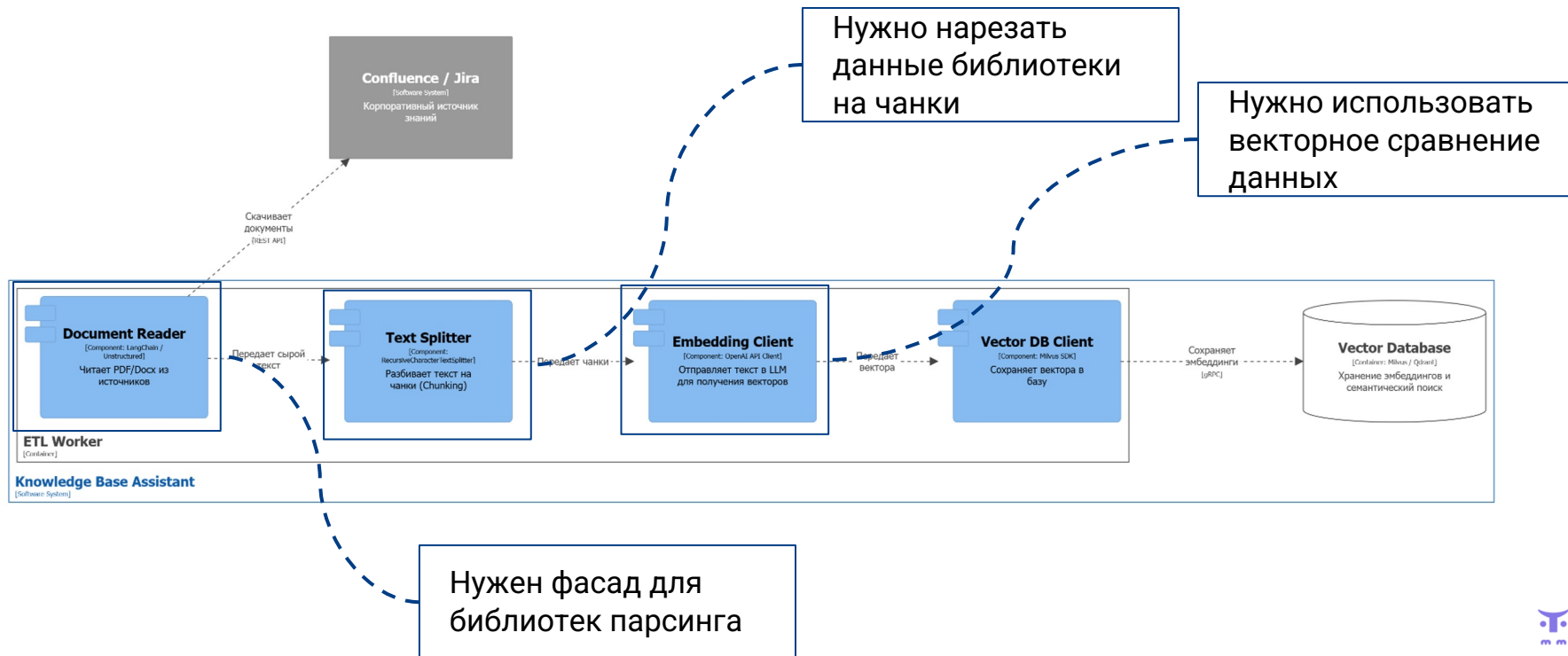
- Взаимодействие элементов диаграммы в динамике
- Диаграмма отвечает на следующие вопросы:
 - Как элементы взаимодействуют друг с другом?
- Основные элементы - элементы конкретного уровня (контейнеры, компоненты)
- Целевая аудитория - архитекторы, разработчики

<https://c4model.com/diagrams/code>



Dynamic View: Internet Banking System - Backend
Summarises how the sign in feature works in the single-page application | Simon Brown | c4model.com | License: CC BY 4.0

Чат-бот. C4 — Level 3

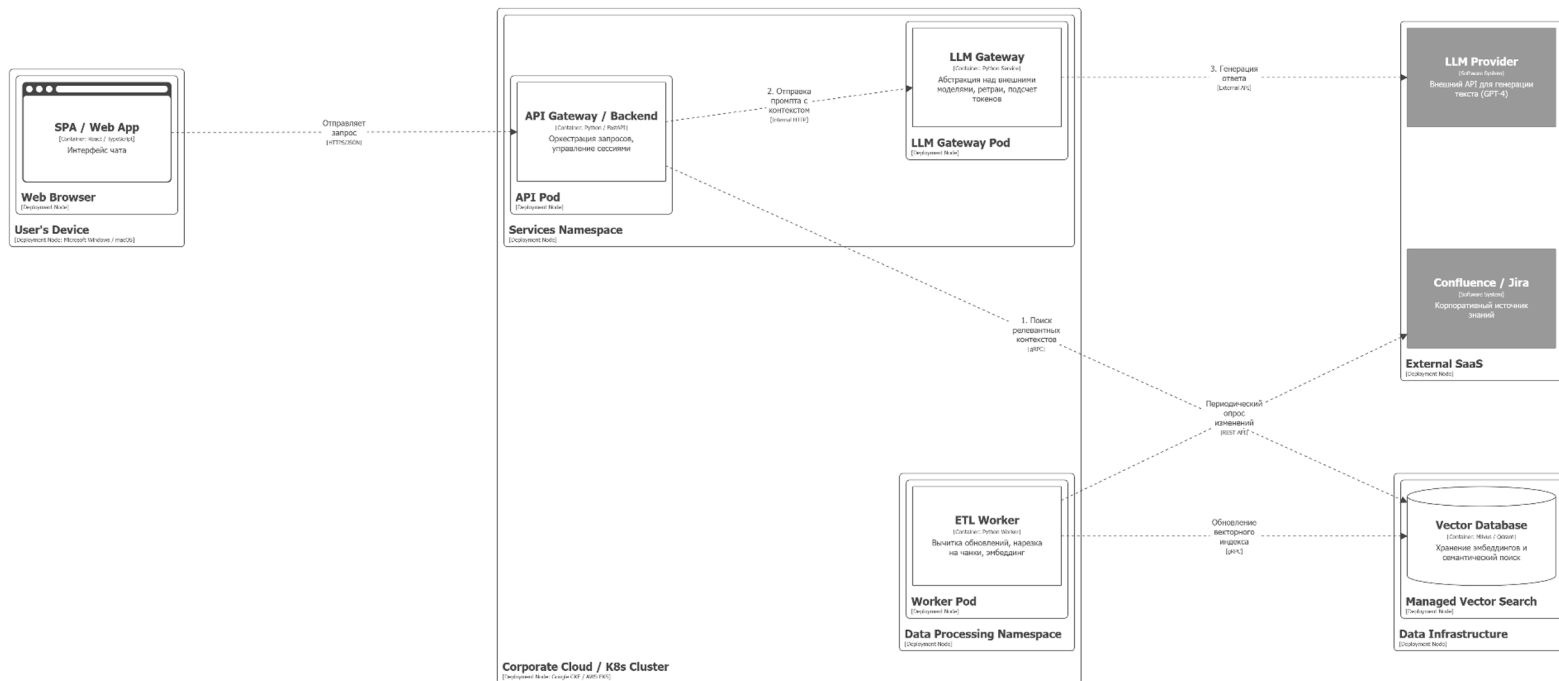


Как понять, что компоненты нарезаны правильно?

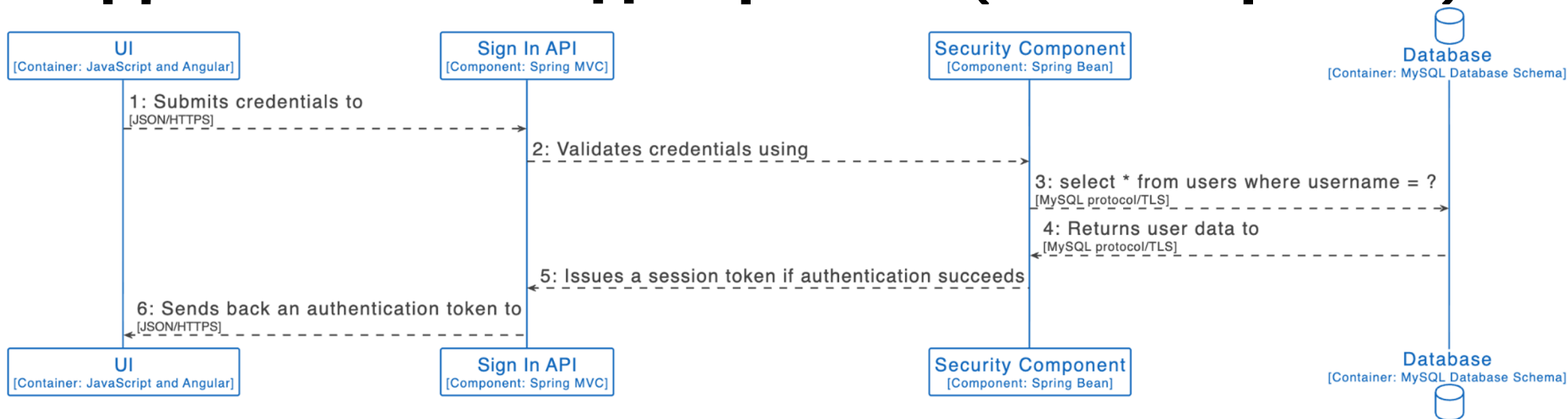
High Cohesion: Компонент Document Reader содержит всю логику работы с PDF (шрифты, картинки, таблицы)

Low Coupling: Компонент RAG Service не знает, что под капотом у Vector DB (Qdrant или Pinecone), он общается через интерфейс

Чат-бот. Deployment



Динамическая диаграмма (UML sequence)



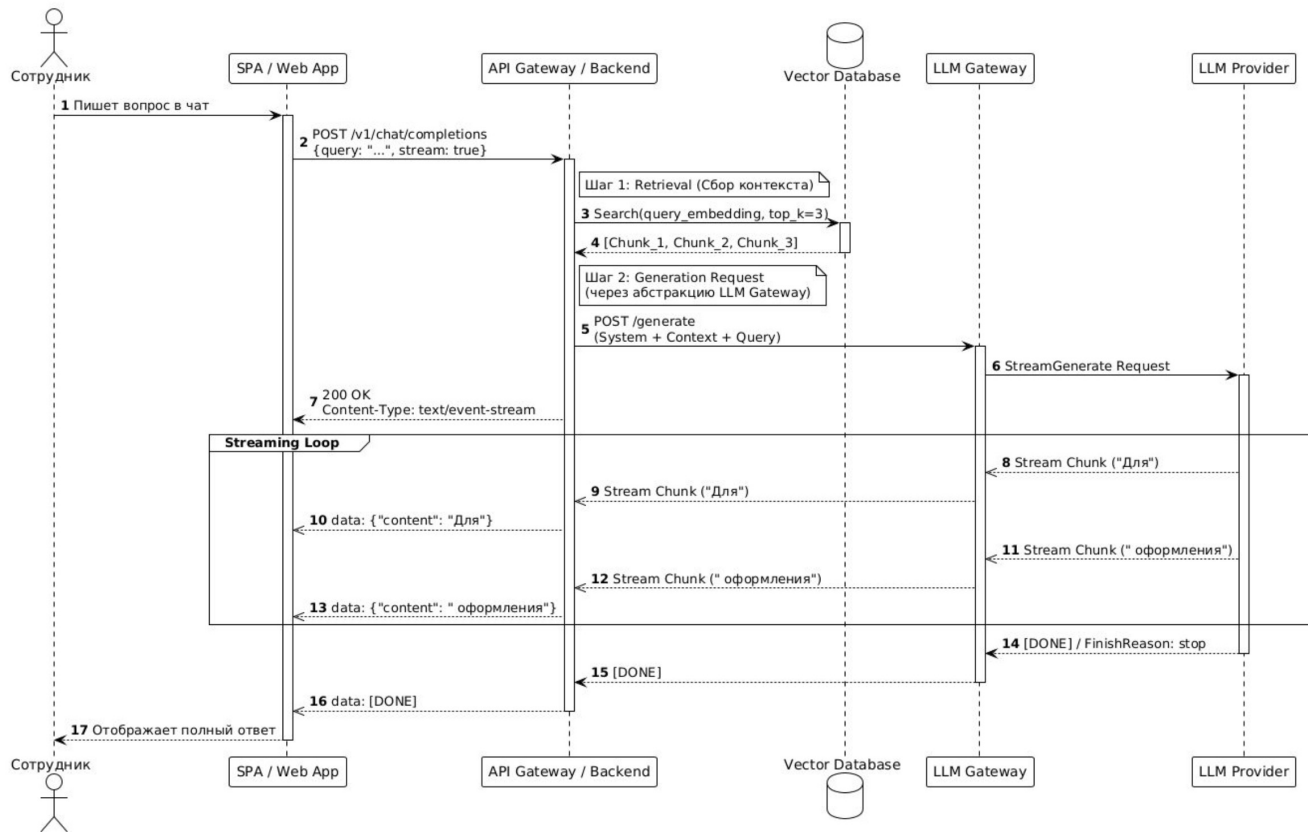
Dynamic View: Internet Banking System - Backend

Summarises how the sign in feature works in the single-page application | Simon Brown | c4model.com | License: CC BY 4.0

- Диаграмма контекста, контейнеров, компонентов и кода позволяет получить статичную картинку того, как выглядит архитектура сейчас.
- Для динамики можно привлекать более подходящие диаграммы

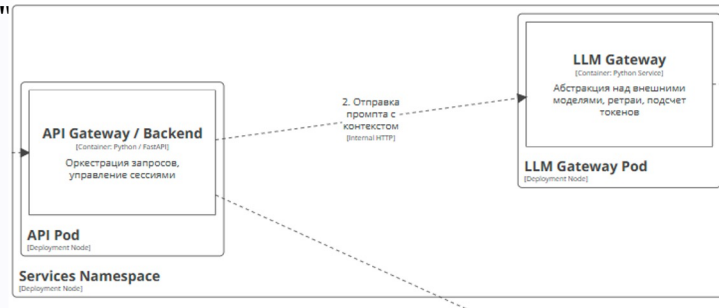
<https://c4model.com/diagrams/code>

Чат-бот. Диаграмма последовательности



Код диаграмм из примеров (деплоймент)

```
model {  
    deploymentEnvironment "Production" {  
        deploymentNode "Corporate Cloud / K8s Cluster" "Корпоративный  
Kubernetes кластер" "Google GKE / AWS EKS" {  
  
            deploymentNode "Services Namespace" "Окружение для синхронных  
сервисов" {  
  
                deploymentNode "API Pod" "Replica Set" {  
                    containerInstance backend  
                }  
                deploymentNode "LLM Gateway Pod" "  
                    containerInstance llmGateway  
                }  
            }  
        }  
    }  
}
```



Что еще забыли при проектировании?



Stateless vs Statefull

Stateless

LLM Service. На входе промпт, на выходе текст. Ничего не помнит. Легко масштабировать.

Statefull

API Gateway/Backend должен помнить состояние сессии (последние запросы чата) - нужно хранилище

API & Contracts

Contract First vs Code First

Contract First

- **Как это:** Пишем спецификацию (OpenAPI YAML / Protobuf)
Генерируем код (стабы) и клиентов.
- **Плюсы:** Параллельная разработка (Front и Back). Стабильный контракт.
- **Минусы:** Нужно знать синтаксис спецификаций. Требуется дисциплины.
- **Вердикт:** Выбор архитектора для Enterprise и интеграций.

Code First

- **Как это:** Пишем код на Python/Java и
Генерируем документацию (Swagger) автоматически.
- **Плюсы:** Быстрый старт, документация всегда соответствует коду.
- **Минусы:** API "гуляет" вместе с рефакторингом кода. Фронтенд ждет, пока бэкенд напишет код.
- **Вердикт:** Подходит для MVP и внутренних сервисов.



OpenAPI (Swagger). Пример

```
openapi: 3.0.0
info:
  title: RAG Assistant API
  version: 1.0.0
paths:
  /chat/completions:
    post:
      summary: Отправить запрос к RAG
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                query:
                  type: string
                  example: "Как оформить отпуск?"
      responses:
        '200':
          description: Успешный ответ
          content:
            application/json:
              schema:
                type: object
                properties:
                  answer:
                    type: string
```

RAG Assistant API 1.0.0 OAS 3.0

default

POST /chat/completions Отправить запрос к RAG

Parameters

No parameters

Request body required

application/json

Example Value | Schema

```
{
  "query": "Как оформить отпуск?"
}
```

Responses

Code	Description	Links
200	Успешный ответ	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{
  "answer": "string"
}
```

Protobuf. Пример

```
syntax = "proto3";
package rag.v1;
service RagService {
    // Метод для чата со стримингом токенов ient отправляет один запрос -> Server возвращает поток ответов (stream)
    rpc ChatStream (ChatRequest) returns (stream ChatResponse);
}
message ChatRequest {
    string session_id = 1;          // ID сессии для контекста
    string user_query = 2;          // Вопрос пользователя: "Как оформить отпуск?"
    int32 max_tokens = 3;          // Лимит токенов (опционально)
}
message ChatResponse {
    string content = 1;             // Очередной кусочек текста (токен)
    bool is_final = 2;             // Флаг окончания генерации
    // Метаданные (возвращаются только в последнем сообщении)
    repeated Source sources = 3;    // Ссылки на документы из Confluence
}
message Source {
    string title = 1;
    string url = 2;
}
```



Версионирование API

- **URI Versioning (Самый популярный):** `POST /v1/chat` → `POST /v2/chat`. Просто и наглядно.
- **Header Versioning (Самый правильный):** `Accept: application/vnd.myapi.v1+json`. Чистый URL, но сложно тестировать в браузере.
- **Query Param (Самый быстрый):** `/chat?version=1`. Часто используется в простых скриптах.



Error Handling: Больше, чем просто 400/500

- Проблема: Фронтенд получает ошибку 500 Server Error или 400 Bad Request и не знает, что сказать пользователю: «Сервер упал» или «У вас кончились лимиты»?
- Решение: Использование стандарта RFC 7807 (Problem Details).
- Анти-паттерн: сервис возвращает 200 OK, а внутри JSON написано {"error": "true"}.

```
HTTP/1.1 400 Bad Request
```

```
Content-Type: application/problem+json
```

```
{  
  "type": "https://example.com/probs/out-of-credit",  
  "title": "You do not have enough credit.",  
  "detail": "Your current balance is 30, but that costs 50.",  
  "instance": "/account/12345/messages/abc",  
  "balance": 30  
}
```



Анти-паттерны C4

Анти-паттерн: "Слоеный пирог"

- Описание проблемы: На одной диаграмме смешаны абстракции разных уровней.
- Пример: Рядом с большим блоком "ERP Система" (Level 1) нарисован маленький класс UserFactory (Level 4) и таблица в БД users_table.
- Почему это плохо: Бизнес не поймет классы, а разработчикам не нужна общая "ERP". Диаграмма становится нечитаемой ни для кого.
- Правило: Один уровень детализации на одну диаграмму.



Анти-паттерн: C4 Container != Docker Container

- Описание проблемы: Архитектор рисует каждый Docker-контейнер как блок на схеме C4.
- Пример: На диаграмме отражены 3 квадратика Nginx, Logstash и App
- Почему это плохо: В C4 "Контейнер" — это логическая исполняемая единица, которая выполняет бизнес-функцию. Nginx часто является деталью развертывания (Deployment Node), а не архитектурным блоком приложения.
- Правило: Если это инфраструктурная "обвязка" — убирайте на Deployment Diagram. Если это код, реализующий логику — оставляйте в Container View.



Анти-паттерн: Неподписанные связи

- Описание проблемы: Стрелочки между блоками есть, но на них нет текста.
- Пример: Сервис А ---> Сервис Б.
- Почему это плохо: Стрелка может значить что угодно: "Отправляет данные", "Запрашивает данные", "Имеет зависимость", "Находится внутри".
- Правило: Любая стрелка должна иметь:
 - Глагол (что делает? — Отправляет отчет).
 - Технологию (как? — HTTPS/JSON).



Вопросы



если есть вопросы



если вопросов нет

**Подведём
итоги занятия**

Теперь вы сможете:

детализировать высокоуровневые
компоненты до уровня конкретных
классов и модулей

проектировать API и протоколы
взаимодействия между сервисами

для того чтобы детализировать
постановку для вашей команды



Ключевые тезисы занятия

1. Contract First — выбор профессионалов Сначала пишем спецификацию (OpenAPI/Proto), договариваемся с фронтендом, и только потом пишем код. Это страхует от интеграционного ада.
2. Архитектура — это не только статика, но и динамика Диаграммы C4 показывают структуру, но для AI-сервисов критически важна динамика (Sequence Diagram), чтобы увидеть порядок вызовов и оптимизировать затраты (например, не вызывать LLM, если база пуста).
3. Надежность проектируется, а не случается В мире нестабильных LLM использование паттернов устойчивости (Circuit Breaker, Retries, Timeouts) — это не опция, а обязанность архитектора.



Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

